Plan de Pruebas

Proyecto: *Anglo Electric*
Fecha: *19/06/2026*
Integrantes: *Fiorella Osorio*
Sección:001D
Profesor: *Claudia Pamela Jelvez*

Información del Proyecto

Organización	Anglo Electric
Proyecto	Plataforma E-Commerce y Dimensionador Solar
Repositorio	Palta2639/Capstone (ramas: Cristopher-Documentacion, Fiorella-Osorio, Main)
Fecha de Inicio	[completar]
Fecha de Término	[completar]
Integrantes	Cristopher Figueroa, Fiorella Osorio, David Valenzuela, Kevin Peña

1. Descripción General del Sistema y Características Claves

Objetivo general

Validar que la plataforma web de Anglo Electric funcione de forma estable, segura y coherente con los requisitos definidos. Para eso se evalúan primero sus componentes por separado (API backend, interfaz frontend y base de datos en la nube) y luego su funcionamiento integrado. Se busca confirmar que el flujo principal, desde el registro del cliente hasta la solicitud de cotización y la gestión administrativa, opera sin errores y protege los datos de los usuarios.

Objetivos específicos

- Verificar que la API desarrollada en FastAPI responda correctamente en sus endpoints de autenticación, catálogo y cotizaciones, aplicando hashing con Bcrypt y tokens JWT.
- Comprobar que la interfaz en React muestre correctamente el catálogo, el carrito de cotización y las vistas de administración, con un comportamiento responsivo.
- Validar que la base de datos PostgreSQL en Neon mantenga la integridad de los datos: claves foráneas, restricciones UNIQUE y consistencia de stock y precios.
- Confirmar que frontend, backend y base de datos funcionen integrados en la rama Main, incluyendo el control de acceso por roles.

2. Propósito del Plan de Pruebas

Este documento establece la estrategia, el alcance y la organización de las pruebas aplicadas a la plataforma Anglo Electric. Como el equipo trabajó de forma modular y en paralelo, el plan evalúa primero cada componente en su rama de desarrollo y después el sistema completo en la rama principal. Así se puede saber si un error nace en un componente específico o en la integración entre ellos.

El documento también define cómo registrar, clasificar y dar seguimiento a los defectos encontrados, y qué condiciones debe cumplir el sistema para aprobarse según la Definition of Done del equipo.

3. Definir Clasificación de los Defectos	
Nivel de Severidad	**Descripción**
Crítico	Falla en el sistema, no es posible seguir con el procesamiento.
Grave	No es posible continuar con el procesamiento de la función seleccionada.
Medio	Funciones restringidas, pero el procesamiento puede continuar.
Leve	Cambio de forma menor.

4. Alcance y estrategia

Requisito / HU	Módulo	Ámbito (rama)	Tipo de prueba
HU-01 Registro e inicio de sesión	Autenticación (Bcrypt + JWT)	A: Cristopher-Documentacion	Unitaria automatizada, caja blanca (pytest)
HU-02 CRUD de productos	API de productos	A: Cristopher-Documentacion	Unitaria / funcional, caja blanca (pytest + Swagger)
HU-03 Catálogo por categorías	Vista de catálogo	B: Fiorella-Osorio	Manual, caja negra
HU-04 Carrito de cotización	Vista de carrito	B: Fiorella-Osorio	Manual, caja negra
Diseño responsivo	Interfaz general	B: Fiorella-Osorio	Manual, caja negra
Integridad de datos	Base de datos Neon	C: Main + BD	Consultas SQL, caja blanca
HU-01 a HU-04 de punta a punta	Sistema completo	D: Integrado	Integración, caja negra y blanca
Control de acceso por roles	Seguridad	D: Integrado	Integración / seguridad
HU-05 y Asistente Virtual	Módulo de Inteligencia Artificial (recomendador KNN, chatbot y agenda de visitas)	D: Integrado (rama feature-mejoras-capstone, en integración a Main)	Integración, caja negra

Módulo de Inteligencia Artificial: el recomendador de kits con el modelo KNN (HU-05), el Asistente Virtual (chatbot) y la agenda de visitas técnicas están a cargo de David Valenzuela, Ingeniero de IA y Datos del equipo. Al momento de ejecutar las Partes A, B y C, este módulo se encontraba en desarrollo en la rama feature-mejoras-capstone y aún no estaba integrado a Main; en Main, el

endpoint /ia/chat solo devuelve un mensaje fijo de prueba. Por esta razón, el módulo se evaluará en la Parte D (casos INT-07 a INT-09) una vez integrado a Main. Fuera del alcance de este plan quedan el dashboard de KPIs (HU-06) y el cambio de estado de cotizaciones (HU-07), que aún no están terminados según las historias de usuario.

5. Entorno y herramientas

- **Backend:** Python 3.10, FastAPI, pytest con TestClient de FastAPI, Swagger (/docs).
- **Frontend: Node.js ≥ 20.9, Next.js 16 (framework de React), navegador Chrome/Brave con DevTools (modo responsivo).**
- **Base de datos:** PostgreSQL en Neon (SQL Editor de la consola).
- **Integración: Docker Desktop 4.99 con WSL 2 y docker compose para levantar el entorno local.**
- **Evidencias:** capturas de pantalla y salida de terminal de pytest.

6. Riesgos

Riesgo	Impacto	Probabilidad	Mitigación	Contingencia
Las pruebas modifican o ensucian los datos reales de la rama *production* de Neon	Alto	Alta	Crear una rama de Neon exclusiva para pruebas y apuntar el .env a ella	Restaurar desde Backup & Restore de Neon
Un usuario cliente accede a funciones de administrador	Muy alto	Baja	Validar el rol en cada endpoint protegido con JWT	Revocar tokens y revisar rol_id en la tabla usuarios
Conflicto de versiones entre passlib y bcrypt impide registrar usuarios	Alto	Media	Fijar versiones en requirements.txt (bcrypt < 4.0.0)	Reinstalar dependencias en un entorno virtual limpio
Las ramas están desactualizadas respecto a Main (hasta 45 commits atrás) y fallan al integrarse	Alto	Alta	Probar cada rama en su estado actual y documentar diferencias antes de unir	Resolver conflictos de merge y repetir las pruebas de la Parte D
El frontend no se comunica con la API (CORS o URL mal configurada)	Alto	Media	Verificar la configuración CORS en FastAPI y la variable de URL	Revisar la consola del navegador y corregir la configuración

			de la API	
Sin conexión a internet no se puede acceder a Neon	Medio	Baja	Tener la cadena de conexión y credenciales listas antes de cada sesión	Reprogramar la sesión de pruebas
El módulo de Inteligencia Artificial no se integra a Main antes de la entrega y no puede probarse en conjunto	Alto	Media	Coordinar con el responsable de IA la fecha de integración y registrar la versión (commit) probada	Probar el módulo en la rama feature-mejoras-capstone y documentarlo como prueba parcial

7. Casos de prueba por parte

Parte A: Backend (rama Cristopher-Documentacion)

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BE-01	Registro de usuario	pytest: POST al endpoint de registro	Nombre, email y contraseña válidos	Código 200/201; la contraseña queda guardada como hash Bcrypt, no en texto plano	Usuario registrado correctamente (Code 200) tras corregir DP-02. La contraseña se almacena como hash Bcrypt ($2b$), no en texto plano.
BE-02	Registro con email duplicado	pytest: registrar dos veces el mismo email	Email ya existente	Error 400; no se crea un segundo usuario	Falló. El segundo registro con el mismo correo devuelve error 500 en vez de 400. La API no valida duplicados y se cae al chocar con la restricción UNIQUE de la base de datos.

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BE-03	Login correcto	pytest: POST al endpoint de login	Credenciales válidas	Devuelve un token JWT válido	Login exitoso (Code 200). El sistema entrega un token JWT válido con token_type "bearer".
BE-04	Login incorrecto	pytest: login con contraseña errónea	Contraseña inválida	Error 401; no se entrega token	Acceso rechazado (Code 401) con el mensaje “Correo o contraseña incorrectos”. No se entrega token. El mensaje genérico evita revelar si el correo existe.
BE-05	Ruta protegida sin token	pytest: llamar a un endpoint protegido sin cabecera Authorization	Sin token	Error 401	Falló. Al eliminar un producto sin token, la API responde 404 (producto no encontrado) en vez de 401. Esto confirma que la ruta no exige autenticación: procesa la solicitud de cualquier persona.
BE-06	Conexión a la BD	pytest: abrir sesión con database.py	DATABASE_URL de prueba	La conexión se establece sin errores	Aprobado. La API se conecta a Neon y responde correctamente (Code 200).
BE-07	Restricción de rol en registro	pytest: registrar un usuario enviando rol_id = 1 (administrador)	Usuario nuevo con rol_id = 1	El sistema rechaza la solicitud o asigna rol cliente; no se crea un administrador	Falló. El registro público acepta rol_id: 1 y crea un usuario administrador (Code 200).

Parte B: Frontend (rama Fiorella-Osorio)

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
FE-01	Levantar la aplicación	npm install y npm run dev / npm start	—	La app carga sin errores en la consola	La aplicación se instala y levanta correctamente con npm run dev (Next.js 16.4, listo en 521 ms). La página de inicio carga sin errores, con navegación, carrusel y secciones visibles.
FE-02	Navegación del catálogo	Manual: entrar al catálogo y filtrar por categoría	Clic en una categoría	Se muestran solo los productos de esa categoría	Parcial. El catálogo carga los 47 productos desde la API y los filtros funcionan correctamente: Paneles Solares muestra 13, Inversores 1, y el filtro "Kits" de Tipo de formato muestra 15 kits. Sin embargo, Baterías, Kits Completos y Kits Off-Grid aparecen vacíos, porque 33 de 47 productos no tienen categoría asignada en la base de datos (ver DP-11).

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
FE-03	Detalle de producto	Manual: clic en una tarjeta de producto	Producto cualquiera	Se ve nombre, precio, descripción e imagen	Aprobado. Al hacer clic en un producto se abre su página de detalle con nombre, SKU, precio final con IVA, imagen, selector de cantidad, botones "Agregar al carro" y "Comprar ahora", y la descripción. El breadcrumb permite volver al inicio. La descripción muestra texto de datos de prueba (ver DP-14).
FE-04	Agregar al carrito	Manual: agregar 2 productos distintos	Clic en “Agregar a cotización”	Ambos aparecen en el carrito con el total correcto	Aprobado. Se agregaron 2 productos distintos y ambos aparecen en el carrito con subtotal correcto ($1.373.800). Al aumentar la cantidad del panel a 5 unidades, el subtotal se recalcula correctamente ($1.953.800) y el contador se actualiza de 2 a 6. Se observan detalles de interfaz menores (ver DP-15 y DP-16).

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
FE-05	Validación de formularios	Manual: enviar registro/login vacío o con email mal escrito	Campos vacíos o inválidos	Se muestran mensajes de error y no se envía el formulario	Aprobado. Al enviar el formulario vacío se muestran mensajes de error en cada campo obligatorio y no se envía la solicitud. El campo nombre bloquea automáticamente los números ("Juan123" queda "Juan"). El formulario informa los requisitos de seguridad de la contraseña: mínimo 9 caracteres, mayúscula, minúscula, número y símbolo.
FE-06	Diseño responsivo	Manual: DevTools en vista móvil y escritorio	Ancho 375 px y 1920 px	La interfaz se adapta sin elementos cortados	Aprobado con observaciones. En vista móvil (375 px) el catálogo se reorganiza en una columna y el carrito se adapta al ancho de la pantalla, manteniendo visibles el subtotal y los botones. En escritorio la interfaz se muestra correctamente. Se observan superposiciones menores en móvil (ver DP-18).
FE-07	Manejo de error sin conexión a la API	Manual: abrir el catálogo con el backend apagado	Backend detenido	La interfaz muestra un mensaje de error de conexión sin caerse	Con el backend apagado, el catálogo muestra un aviso de error de conexión y un botón “Reintentar”. La aplicación no se cae y registra el error en la consola.

Parte C: Main + base de datos Neon

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BD-01	Estructura vs. diseño	Comparar tablas y columnas de Neon con el Documento de Diseño	Esquema public	Las tablas y columnas coinciden con el modelo ER, o las diferencias quedan justificadas	Parcial. La base de datos contiene las 8 tablas del modelo entidad-relación (roles, usuarios, categorias, productos, cotizaciones, pedidos, detalle_pedidos y agenda) con sus columnas. Sin embargo, existen 5 tablas y 1 vista no documentadas (citas_visita, turnos_visita, costos_productos, cotizaciones_knn_detalle, pedidos_registro y vista_kpis_financieros), y la columna es_kit de productos tampoco figura en el diseño. Además, el texto de la sección 6.2 y el diagrama de clases mencionan la tabla detalle_cotizaciones y la columna potencia_w, que no existen en la base (ver DP-20).
BD-02	Email único	SQL: insertar dos usuarios con el mismo email	Email repetido	La BD rechaza el segundo registro	Aprobado. La base de datos acepta el primer registro y rechaza el segundo con el mismo correo, mostrando el error "duplicate key value violates unique constraint usuarios_email_key". La restricción UNIQUE sobre el campo email funciona correctamente. Esto confirma que el error 500 de BE-02 (DP-03) ocurre porque el backend no controla este rechazo de la base de datos, no porque falte la restricción.

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BD-03	Claves foráneas	SQL: insertar un producto con categoria_id inexistente	categoria_id = 999	La BD rechaza la inserción	Aprobado. La base de datos rechaza la inserción de un producto con categoria_id = 999, mostrando el error "violates foreign key constraint productos_categoria_id_fkey", porque esa categoría no existe. La clave foránea entre productos y categorías funciona correctamente. Sin embargo, la columna acepta valores vacíos (NULL), lo que permite crear productos sin categoría (ver DP-11).
BD-04	Contraseñas cifradas	SQL: SELECT password_hash FROM usuarios	—	Todos los valores son hashes Bcrypt (empiezan con $2b$)	Aprobado. Los 17 usuarios registrados a través del sistema tienen su contraseña almacenada como hash Bcrypt ($2b$), sin contraseñas en texto plano. La verificación se hizo contando registros, sin exponer los hashes. Se observa que la base de datos no impide guardar contraseñas sin cifrar (en BD-02 aceptó el valor "hash_prueba"), por lo que el cifrado depende completamente del backend.

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BD-05	Calidad de datos de productos	SQL: buscar productos con stock negativo, precio 0, categoría o potencia nula	Tabla productos	No existen registros inválidos, o quedan reportados como defecto	Parcial. De 47 productos, no hay registros con stock negativo ni precios inválidos. Sin embargo, la calidad de los datos es baja: 40 productos son datos simulados de prueba, 33 no tienen categoría, 46 no tienen imagen, 12 no tienen potencia registrada, 6 no tienen descripción y 6 tienen stock en 0. Estos datos explican los defectos observados en el frontend (DP-11, DP-12, DP-13 y DP-14) y afectan el filtro por potencia y el futuro recomendador KNN, que depende de este dato (ver DP-21).
BD-06	Levantar Main con Docker	docker compose up en la rama Main	docker-compose.yml	Backend y frontend se levantan sin errores	Falló. Docker Desktop se instaló (requirió activar WSL 2) y docker compose up se ejecuta sin errores, levantando 2 contenedores: PostgreSQL 16 local (puerto 5432) y pgAdmin (puerto 5050). Sin embargo, no se levantan el backend ni el frontend: el archivo no define esos servicios y el repositorio no contiene Dockerfile, a diferencia de lo indicado en la sección 6.6 del Documento de Diseño. Además, la base local no se utiliza porque el sistema se conecta a Neon (ver DP-22 y DP-23).

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
BD-07	Scripts de base de datos versionados	Revisar Schema.sql y Seed.sql de la rama Kevin-Peña y compararlos con la base en Neon	Archivos de la carpeta database	Los scripts permiten recrear la base actual completa y con datos coherentes	Parcial. La rama Kevin-Peña contiene Schema.sql, Seed.sql, diagrama DBML, colección Postman, .env.example sin credenciales reales y .gitignore. Sin embargo, el esquema documenta solo 8 tablas (faltan 5 tablas y 1 vista existentes en Neon), el Seed incluye un producto ajeno al negocio ("Taladro Inalámbrico 20V") y define 4 categorías mientras el frontend usa 5. Los scripts se enviaron a Main mediante el Pull Request #3 (ver DP-24).

Parte D: Pruebas integradas

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
INT-01	Registro de punta a punta	Registrarse desde el frontend y revisar la tabla usuarios en Neon	Usuario nuevo	El usuario aparece en la BD con la contraseña cifrada	
INT-02	Login y sesión	Iniciar sesión desde el frontend	Credenciales válidas	Se recibe el token y la interfaz muestra la sesión iniciada	
INT-03	Catálogo desde la BD	Cambiar el precio de un producto en Neon y recargar el catálogo	Precio modificado	El frontend muestra el precio actualizado	
INT-04	CRUD de administrador	Crear un producto desde el panel admin	Datos válidos de producto	El producto queda en Neon y se ve en el catálogo	
INT-05	Control de roles	Con sesión de cliente, intentar entrar al panel admin	Token de rol cliente	Acceso denegado (403 o redirección)	

ID	Funcionalidad	Procedimiento	Datos de entrada	Resultado esperado	Resultado obtenido
INT-06	Solicitud de cotización	Agregar productos al carrito y enviar la solicitud	2 productos	Se crea el registro en cotizaciones y su detalle en la BD	
INT-07	Recomendador de kits con IA (KNN)	Ingresar consumo mensual en kWh y presupuesto en el Cotizador IA y solicitar la recomendación	Consumo y presupuesto válidos	El sistema devuelve un kit solar acorde al consumo y presupuesto, con su precio	Pendiente: módulo en integración a Main
INT-08	Asistente Virtual (chatbot)	Abrir el Asistente Virtual y realizar una consulta sobre energía solar	Pregunta del usuario	El asistente responde de forma coherente a la consulta, no con un mensaje fijo	Pendiente: módulo en integración a Main
INT-09	Agenda de visita técnica	Agendar una visita técnica desde la plataforma y revisar las tablas citas_visita y turnos_visita en Neon	Fecha, dirección y comuna	La visita queda registrada en la base de datos y se muestra al usuario	Pendiente: módulo en integración a Main

Resultado de las Pruebas (Cobertura)

Parte A – Backend: En la Parte A se ejecutaron 7 casos de prueba sobre el backend, combinando pruebas manuales en Swagger y pruebas automatizadas con pytest. Se aprobaron 4 casos (57%): registro con contraseña cifrada, login correcto, login incorrecto y conexión a la base de datos. Fallaron 3 casos relacionados con validación y seguridad: correos duplicados, rutas sin protección por token y asignación libre de roles. La autenticación funciona correctamente, pero el token JWT generado aún no se utiliza para proteger las rutas.

Parte B – Frontend: Se ejecutaron 7 casos de prueba manuales (caja negra) sobre la interfaz desarrollada en Next.js. Se aprobaron 6 casos y 1 resultó parcial. La aplicación se instala y levanta correctamente, maneja bien la falta de conexión con el backend, muestra el detalle de productos, calcula correctamente el carrito, valida los formularios y se adapta a pantallas móviles. El caso parcial corresponde al filtro por categorías, que funciona bien en el frontend pero muestra categorías vacías porque 33 de 47 productos no tienen categoría asignada en la base de datos. Los principales hallazgos son 9 vulnerabilidades en dependencias (una crítica, en Next.js), la validación de contraseñas solo del

lado del cliente y diferencias entre el carrito implementado y la HU-04, que pide solicitud de cotización en vez de pago.

Parte C – Base de datos y Main: Se ejecutaron 7 casos sobre la base de datos Neon (rama de pruebas) y la rama Main, mediante consultas SQL, inspección de scripts y Docker. Se aprobaron 3 casos (unicidad de email, claves foráneas y contraseñas cifradas con Bcrypt), 3 resultaron parciales (estructura frente al diseño, calidad de datos y scripts versionados) y 1 falló (Docker). La base protege correctamente la integridad y la seguridad de las credenciales, pero la calidad de los datos es baja: 40 de 47 productos son simulados, 33 no tienen categoría y 46 no tienen imagen. Además, el Documento de Diseño y los scripts no reflejan la estructura real de la base, y la configuración de Docker no levanta la aplicación como indica el diseño.

Detalle del resultado de BD-06 (Docker):

Según la sección 6.6 del Documento de Diseño, el proyecto debía empaquetarse en contenedores Docker para que, con un solo comando (docker compose up), se levantaran el backend (FastAPI, puerto 8000) y el frontend (Next.js, puerto 3000), conectados a la base de datos en Neon. De esta forma, el sistema funcionaría igual en cualquier computador sin instalar dependencias manualmente.

Al ejecutar docker compose up en la rama Main, Docker funcionó correctamente, pero levantó solo dos contenedores: anglo_postgres, una base de datos PostgreSQL local y vacía, y anglo_pgadmin, una herramienta para administrarla. Ninguno de los dos corresponde a la aplicación: el archivo docker-compose.yml no incluye servicios para el backend ni el frontend, y el repositorio no contiene archivos Dockerfile, necesarios para empaquetarlos. Además, la base local no se utiliza, ya que el sistema se conecta a Neon, y las contraseñas de ambos servicios están escritas en texto plano en el archivo.

Por lo tanto, la prueba se considera fallida: el problema no está en la herramienta Docker, sino en que la configuración del proyecto está desactualizada y corresponde a una etapa anterior, cuando se usaba una base de datos local. Para corregirlo, el equipo debe crear un Dockerfile para el backend y otro para el frontend, agregarlos como servicios en docker-compose.yml, retirar o dejar como opcionales los servicios de base local y trasladar las credenciales a un archivo .env (ver DP-22).

Adicionalmente, para ejecutar Docker en Windows fue necesario instalar Docker Desktop y activar WSL 2 (característica Virtual Machine Platform) mediante el comando wsl --install y un reinicio del equipo. Este requisito no está indicado en el README del proyecto, por lo que cualquier integrante o evaluador encontraría el mismo obstáculo (ver DP-23).

Defectos Encontrados en las Pruebas

ID Defecto	ID CP	Descripción	Tipo	Severidad	Estado
DP-01	BE-06	La rama no incluye requirements.txt; el servidor no inicia por falta del driver psycopg requerido por la conexión a Neon (ModuleNotFoundError).	Configuración	Grave	Corregido (instalación manual)
DP-02	BE-01	El registro de usuario falla con error 500 (SSL connection has been closed unexpectedly) cuando la base de datos Neon entra en suspensión por inactividad, ya que el backend reutiliza conexiones cerradas.	Configuración / Conexión	Grave	Corregido (se agregó pool_pre_ping=True en database.py)
DP-03	BE-02	Al registrar un correo ya existente, la API devuelve error 500 en lugar de un mensaje controlado (400), porque no verifica duplicados antes de guardar.	Validación	Medio	Abierto
DP-04	BE-05	Los endpoints de administración (eliminar y crear productos) no exigen token JWT. Cualquier persona puede modificar el catálogo.	Seguridad	Grave	Abierto
DP-05	BE-07	El registro público permite que el usuario elija su rol, lo que permite crear cuentas de administrador sin autorización.	Seguridad	Grave	Abierto
DP-06	General	Las credenciales de Neon y la SECRET_KEY de JWT están escritas directamente en el código de un repositorio público.	Seguridad	Grave	Abierto

ID Defecto	ID CP	Descripción	Tipo	Severidad	Estado
DP-07	General	pytest muestra 11 advertencias por funciones obsoletas (declarative_base, class Config de Pydantic, datetime.utcnow). No afectan el funcionamiento actual, pero pueden fallar en versiones futuras.	Mantenibilidad	Leve	Abierto
DP-08	FE-01	npm audit reporta 9 vulnerabilidades en dependencias del frontend (7 moderadas, 1 alta, 1 crítica). La crítica corresponde a la versión instalada de Next.js (ejecución remota de código y denegación de servicio); la alta, a follow-redirects; las moderadas, a minimatch y paquetes de typescript-eslint.	Seguridad	Grave	Abierto
DP-09	FE-07	Ante un error de conexión, además del aviso se muestra "No encontramos coincidencias… filtros seleccionados", lo que puede confundir al usuario, porque el problema es la conexión y no los filtros.	Usabilidad	Leve	Abierto
DP-10	FE-01	La consola advierte que la imagen /logo-anglo.png tiene modificado solo el ancho o el alto, lo que puede deformar el logo.	Interfaz	Leve	Abierto
DP-11	FE-02	33 de 47 productos no tienen categoría asignada en la base de datos (categoria_id vacío). Por eso las categorías Baterías, Kits Completos y Kits Off-Grid aparecen vacías, aunque existen baterías y kits en el catálogo.	Datos	Medio	Abierto

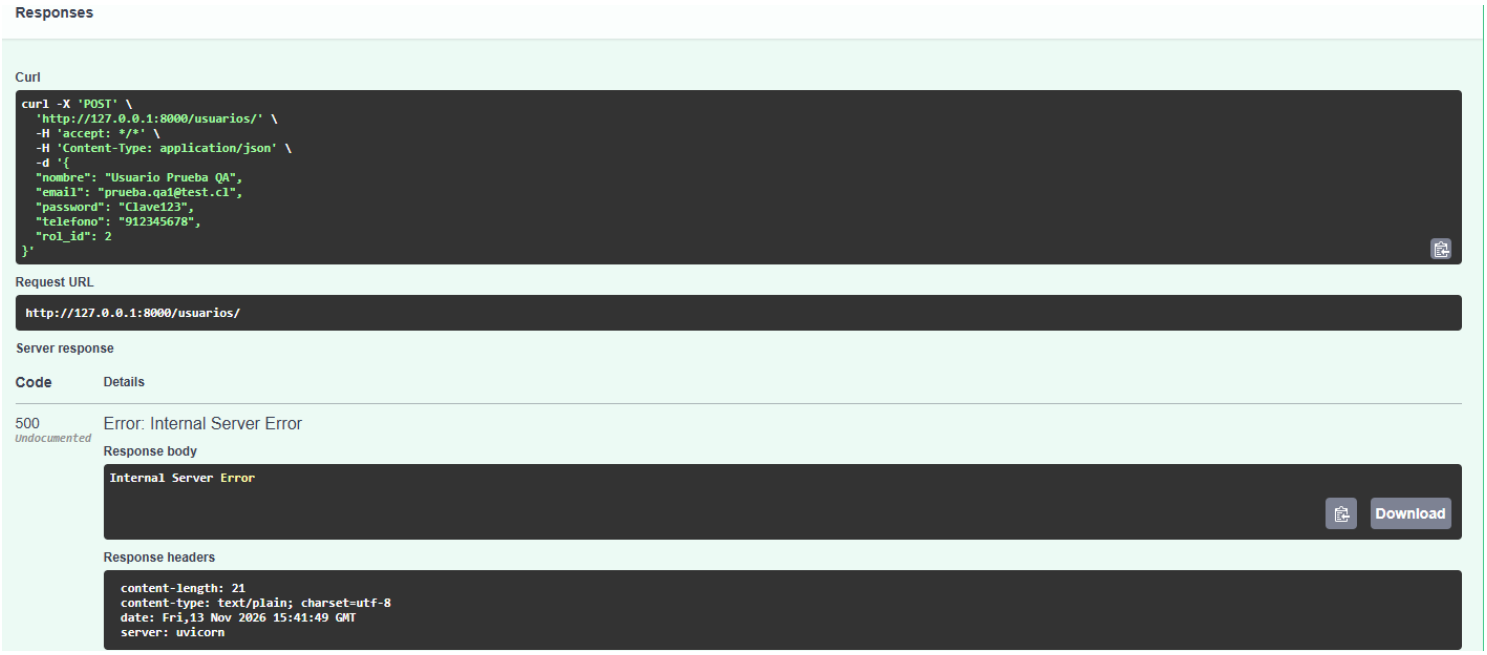
ID Defecto	ID CP	Descripción	Tipo	Severidad	Estado
DP-12	FE-02	El producto "Panel Solar Monocristalino 450W" no muestra su imagen; aparece solo el texto alternativo.	Interfaz	Leve	Abierto
DP-13	FE-02	Todos los productos usan la misma imagen de panel solar, incluso baterías, inversores y kits, lo que puede confundir al cliente.	Datos / Interfaz	Leve	Abierto
DP-14	FE-03	La rama production de la base de datos contiene productos de prueba: 40 de 47 productos tienen en su descripción el texto "SIMULADO PARA PRUEBAS; precio CLP y stock ficticios", visible para el cliente (confirmado en BD-05).	Datos	Medio	Abierto
DP-15	FE-04	En el carrito los productos aparecen como “Sin foto”, aunque en el catálogo sí muestran imagen. Además, cada ítem muestra el precio unitario y no el subtotal de la línea (5 paneles muestran $145.000), lo que puede confundir al cliente.	Interfaz / Usabilidad	Leve	Abierto
DP-16	FE-04	El flujo usa “Agregar al carro” y “Proceder al Pago”, pero según la HU-04 y el Documento de Inicio, el carrito debe generar una **solicitud de cotización**, y los pagos reales (Webpay) están fuera del alcance del proyecto.	Requisitos	Medio	Abierto
DP-17	FE-05 / BE-01	Los requisitos de contraseña (9 caracteres, mayúscula, número y símbolo) solo se validan en el frontend. El backend acepta contraseñas débiles como “Clave123” si se llama directamente a la API.	Seguridad	Medio	Abierto

ID Defecto	ID CP	Descripción	Tipo	Severidad	Estado
DP-18	FE-06	En vista móvil, el nombre de producto largo se superpone al ícono de eliminar en el carrito, y el botón flotante “Asistente Virtual” tapa parte del contenido inferior.	Interfaz	Leve	Abierto
DP-19	General	Las pruebas de las Partes A y B se ejecutaron sobre la rama production de Neon, porque no existía una rama de pruebas. Se crearon usuarios de prueba en la base real, incluidos usuarios con rol administrador generados por BE-07.	Proceso / Datos	Medio	Abierto
DP-20	BD-01	El Documento de Diseño está desactualizado respecto a la base de datos: no documenta 5 tablas y 1 vista existentes, ni la columna es_kit. Además, el texto de la sección 6.2 y el diagrama de clases mencionan la tabla detalle_cotizaciones y la columna potencia_w, que no existen (la real es potencia_watts).	Documentación	Medio	Abierto
DP-21	BD-05	Datos incompletos en la tabla productos: 46 de 47 sin imagen, 12 sin potencia registrada y 6 sin descripción. La falta de potencia afecta el filtro del catálogo y el funcionamiento del recomendador KNN.	Datos	Medio	Abierto
DP-22	BD-06	El docker-compose.yml solo levanta PostgreSQL local y pgAdmin; no incluye servicios para backend ni frontend y no existen Dockerfile, contradiciendo la sección 6.6 del Documento de Diseño. Además, las contraseñas de Postgres y pgAdmin están escritas en texto plano en el archivo.	Configuración / Despliegue	Medio	Abierto
DP-23	BD-06	El README no indica que se requiere Docker Desktop y WSL 2 para usar docker compose; la instalación falló hasta activar manualmente la característica "Virtual Machine Platform" de Windows.	Documentación	Leve	Abierto

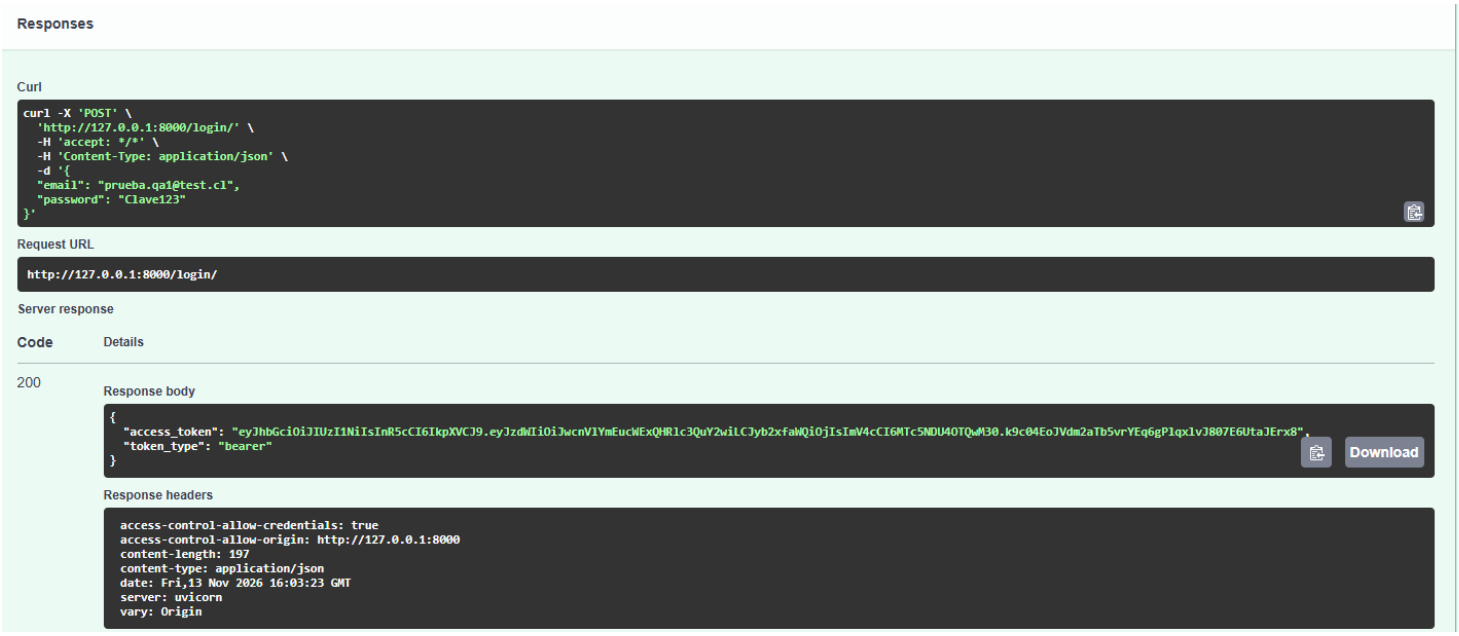
ID Defecto	ID CP	Descripción	Tipo	Severidad	Estado
DP-24	BD-07	Los scripts de base de datos (Schema.sql y README) documentan 8 tablas, pero la base en Neon tiene 13 tablas y 1 vista. El Seed incluye un producto ajeno al negocio ("Taladro Inalámbrico 20V") y solo 4 categorías, mientras el frontend usa 5.	Datos / Documentación	Leve	Abierto

Anexos

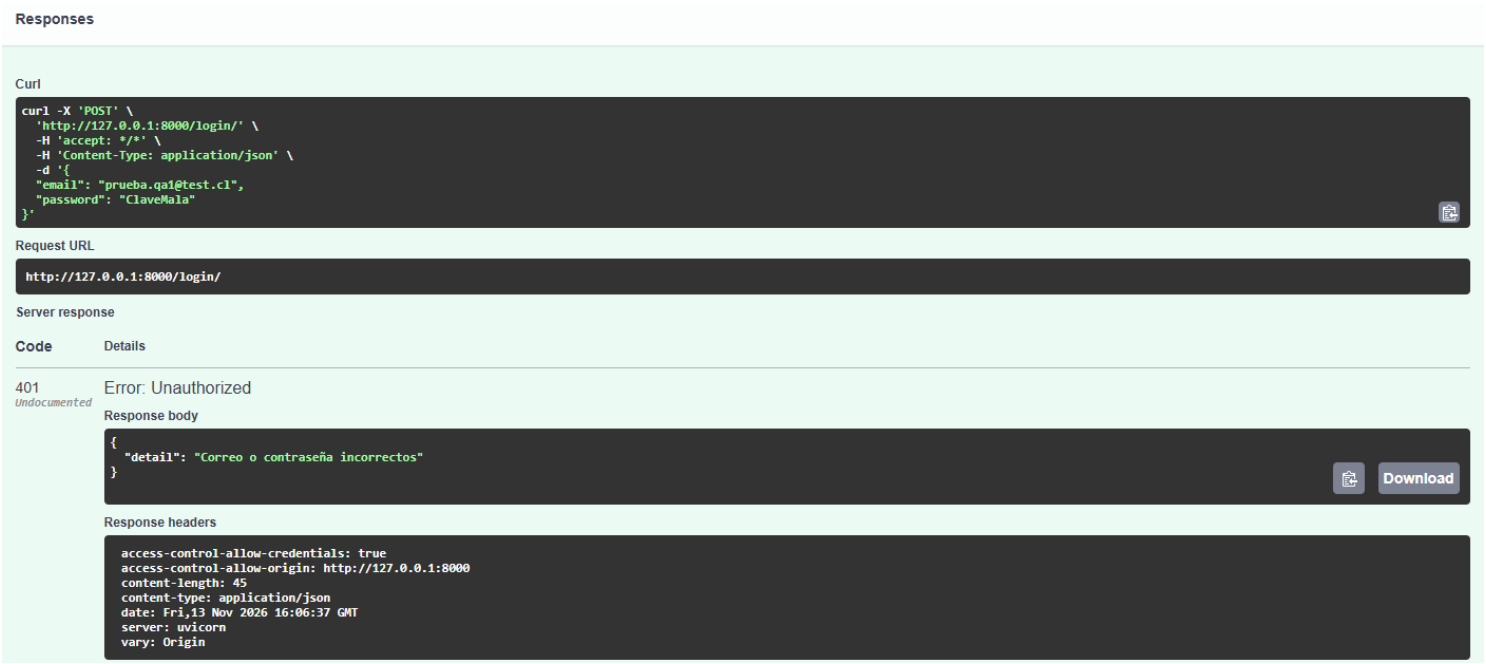
Anexo 1 – DP-02 / BE-01: Error 500 al registrar usuario en Swagger por conexión cerrada con Neon.



Anexo 2 – BE-03: Login correcto en Swagger, el sistema entrega token JWT (Code 200).



Anexo 3 – BE-04: Login con contraseña incorrecta rechazado (Code 401).



Anexo 4 – BE-01 a BE-07: Ejecución de pruebas automatizadas con pytest (resultados).

```
(venv) PS C:\Users\Nox\Desktop\TRABAJO\parteA_backend> pytest test_backend.py -v
rootdir: C:\Users\Nox\Desktop\TRABAJO\parteA_backend
plugins: anyio-4.16.0
collected 7 items

test_backend.py::test_registro_usuario PASSED                                                
```

Anexo 6 – Resumen final de pytest: 4 aprobadas, 3 fallidas.

```
schemas.py:107
  C:\Users\Nox\Desktop\TRABAJO\parteA_backend\schemas.py:107: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Py
dantic V2 Migration Guide at https://errors.pydantic.dev/2.14/migration/
    class AgendaResponse(AgendaCreate):

test_backend.py::test_login_correcto
  C:\Users\Nox\Desktop\TRABAJO\parteA_backend\main.py:47: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in
 UTC: datetime.datetime.now(datetime.UTC).
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=========================================================================== short test summary info ============================================================================
FAILED test_backend.py::test_correo_duplicado - assert 500 == 400
FAILED test_backend.py::test_eliminar_sin_token - assert 404 == 401
FAILED test_backend.py::test_registro_como_admin - assert (200 != 200 or 1 != 1)
================================================================== 3 failed, 4 passed, 11 warnings in 7.42s ==================================================================
(venv) PS C:\Users\Nox\Desktop\TRABAJO\parteA_backend>
```

Anexo 7 – DP-08: npm install reporta 9 vulnerabilidades.

```
9 vulnerabilities (7 moderate, 1 high, 1 critical)

To address all issues, run:
  npm audit fix
PS C:\Users\Nox\Desktop\TRABAJO\parteB_frontend\frontend>
```

Anexo 8 – DP-08: Detalle de vulnerabilidades según npm audit.

```
PS C:\Users\Nox\Desktop\TRABAJO\parteB_frontend\frontend> npm audit
# npm audit report

follow-redirects  1.16.0 - 1.16.1
Severity: high
follow-redirects: Cross-realm sensitiveHeaders arrays are discarded and custom credentials can follow redirects - https://github.com/advisories/GHSA-mrwp-f3q8-3g9p
fix available via `npm audit fix`
node_modules/follow-redirects

minimatch  7.0.0 - 11.0.0
Severity: moderate
minimatch has potential denial of service via globparts amplification - https://github.com/advisories/GHSA-g3hv-75wg-m249
fix available via `npm audit fix`
node_modules/@typescript-eslint/typescript-estree/node_modules/minimatch
  @typescript-eslint/typescript-estree  6.16.0 - 8.72.2
  Depends on vulnerable versions of minimatch
  node_modules/@typescript-eslint/typescript-estree
    @typescript-eslint/parser  6.16.0 - 8.72.2
    Depends on vulnerable versions of @typescript-eslint/typescript-estree
    node_modules/@typescript-eslint/parser
      typescript-eslint  <=8.72.2
      Depends on vulnerable versions of @typescript-eslint/eslint-plugin
      Depends on vulnerable versions of @typescript-eslint/parser
      Depends on vulnerable versions of @typescript-eslint/typescript-estree
      Depends on vulnerable versions of @typescript-eslint/utils
      node_modules/typescript-eslint
    @typescript-eslint/type-utils  6.16.0 - 8.72.2
    Depends on vulnerable versions of @typescript-eslint/typescript-estree
    Depends on vulnerable versions of @typescript-eslint/utils
    node_modules/@typescript-eslint/type-utils
      @typescript-eslint/eslint-plugin  6.16.0 - 8.72.2
      Depends on vulnerable versions of @typescript-eslint/type-utils
      Depends on vulnerable versions of @typescript-eslint/utils
      node_modules/@typescript-eslint/eslint-plugin
    @typescript-eslint/utils  6.16.0 - 8.72.2
    Depends on vulnerable versions of @typescript-eslint/typescript-estree
    node_modules/@typescript-eslint/utils

next  16.0.0 - 16.4.0
Severity: critical
Next.js: Unauthenticated Remote Code Execution in Image Optimization API when SVG files are used - https://github.com/advisories/GHSA-2464-fhr4-w9xw
Next.js: Denial of Service in App Router using Server Actions - https://github.com/advisories/GHSA-p3m5-59mp-jv5r
Next.js: Unauthenticated Remote Code Execution in `use cache` functions returning typed arrays - https://github.com/advisories/GHSA-673q-mjm6-r5xr
fix available via `npm audit fix`
node_modules/next
```

Anexo 9 – FE-01: Frontend levantado con npm run dev.

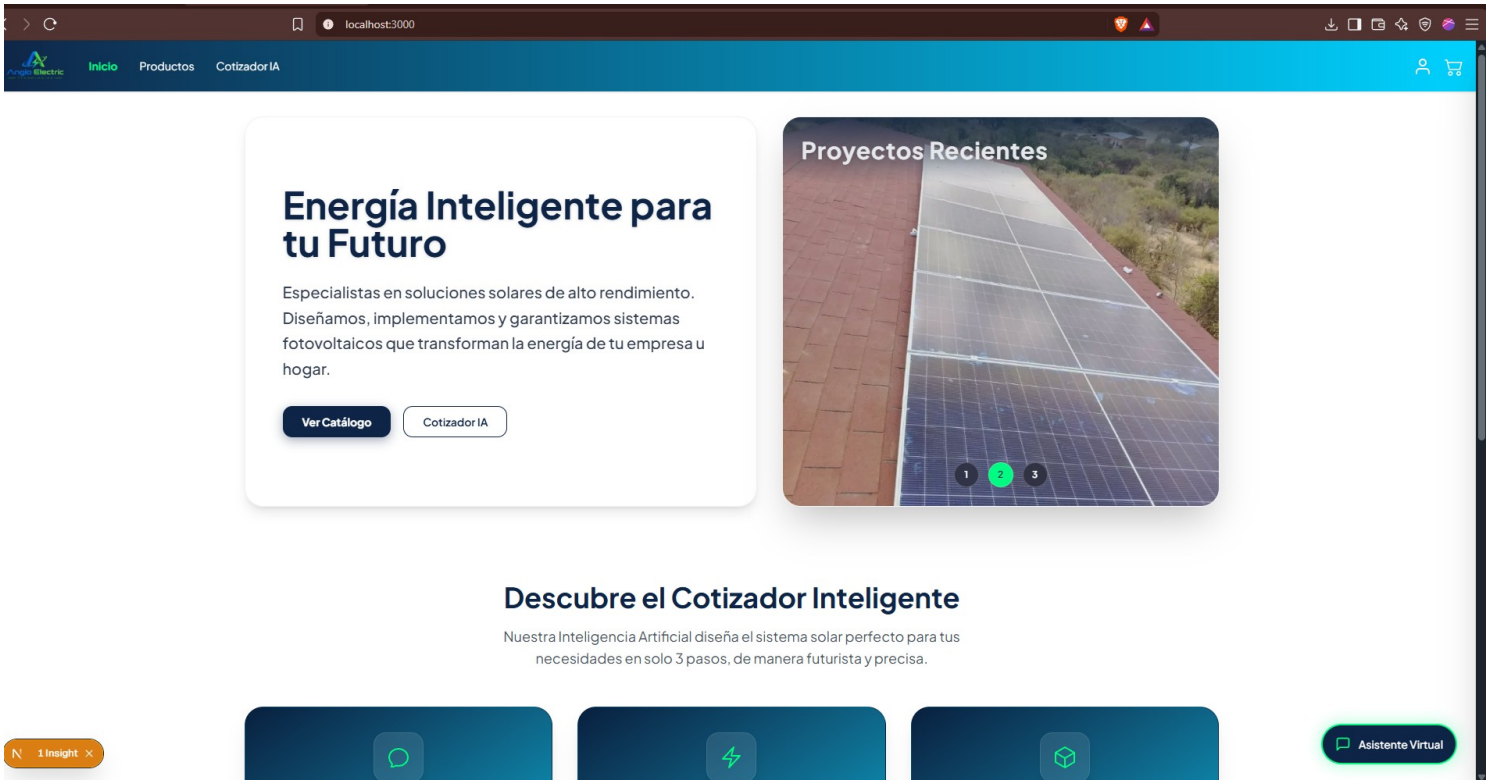
```
PS C:\Users\Nox\Desktop\TRABAJO\parteB_frontend\frontend> npm run dev

> anglo-electric-1@0.1.0 dev
> next dev

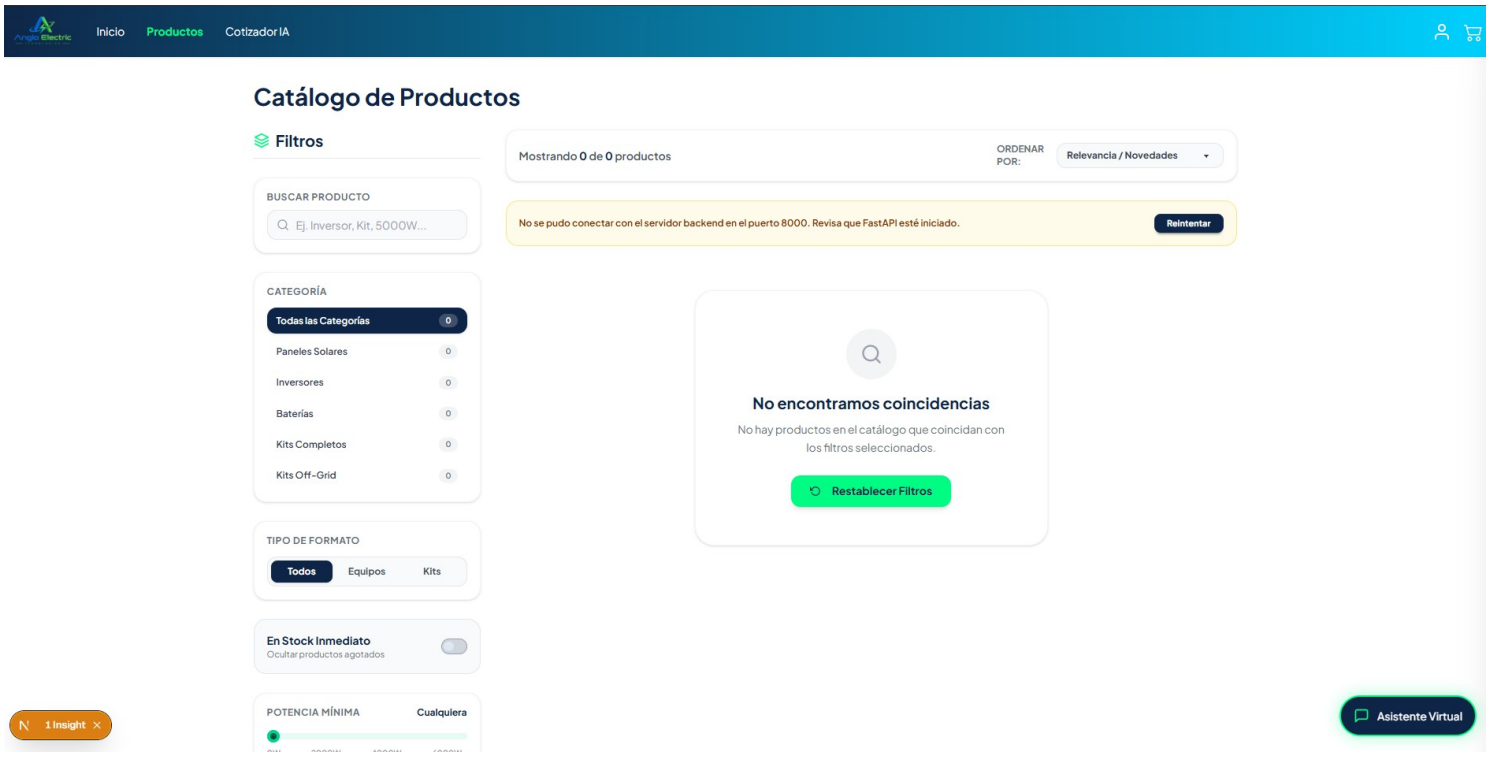
▲ Next.js 16.4.0 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.1.5:3000
✓ Ready in 521ms
✓ Running next.config.mjs took 22ms

 GET / 200 in 469ms (next.js: 170ms, application-code: 299ms)
```

Anexo 10 – FE-01: Página de inicio cargada en localhost:3000.



Anexo 11 – FE-07 / DP-09: Catálogo sin conexión al backend, muestra aviso y botón Reintentar.



Anexo 12 – FE-07 / DP-10: Registro del error de conexión y advertencia del logo en la terminal.

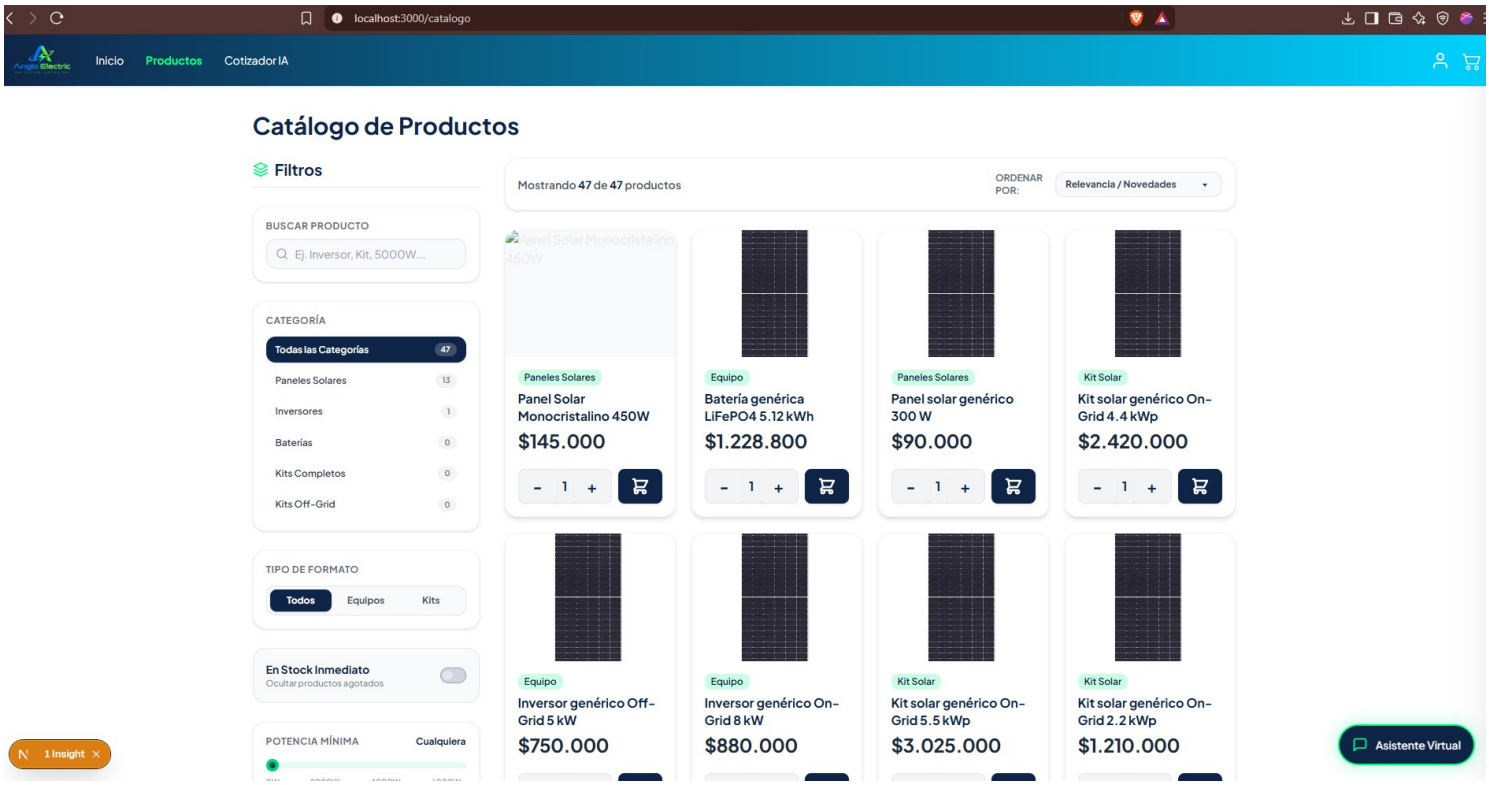
```
PS C:\Users\Nox\Desktop\TRABAJO\parteB_frontend\frontend> npm run dev

> anglo-electric-1@0.1.0 dev
> next dev

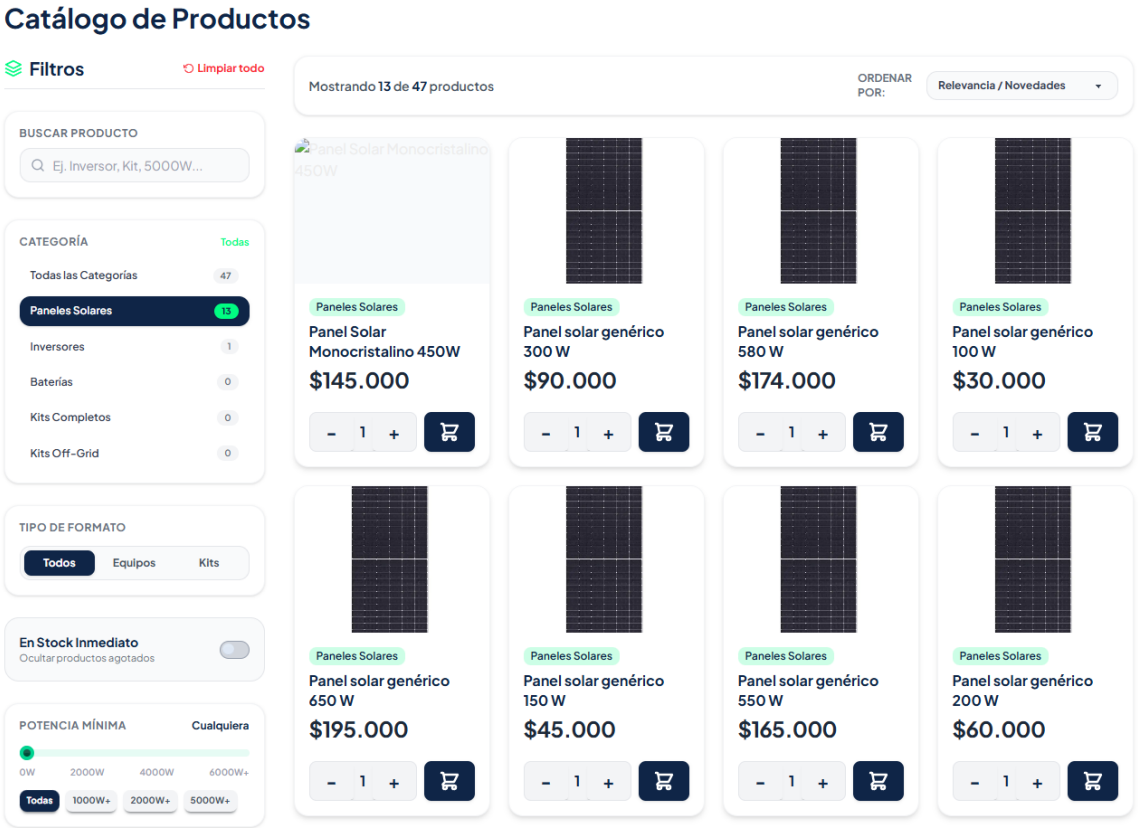
▲ Next.js 16.4.0 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.1.5:3000
✓ Ready in 521ms
✓ Running next.config.mjs took 22ms

 GET / 200 in 469ms (next.js: 170ms, application-code: 299ms)
[browser] Backend no disponible temporalmente para productos destacados: Network Error (src/app/(tienda)
[browser] Backend no disponible temporalmente para productos destacados: Network Error (src/app/(tienda)
[browser] Image with src "/logo-anglo.png" has either width or height modified, but not the other. If you use CSS to change the size of your image, also include the styles 'wid
th: "auto"' or 'height: "auto"' to maintain the aspect ratio.
 GET /catalogo 200 in 33ms (next.js: 10ms, application-code: 22ms)
[browser] Backend FastAPI no disponible o reconectando: Network Error (src/components/VistaCatalogo.jsx:35:21)
[browser] Backend FastAPI no disponible o reconectando: Network Error (src/components/VistaCatalogo.jsx:35:21)
```

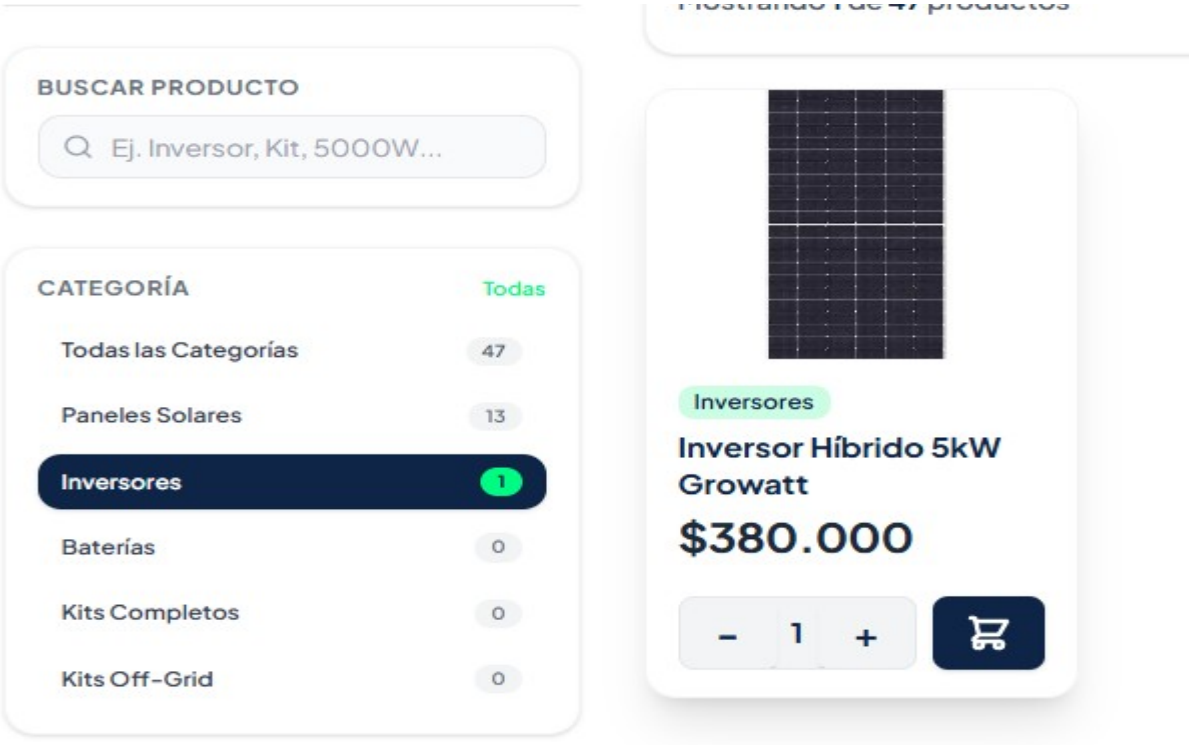
Anexo 13 – FE-02: Catálogo completo con 47 productos cargados desde la API.



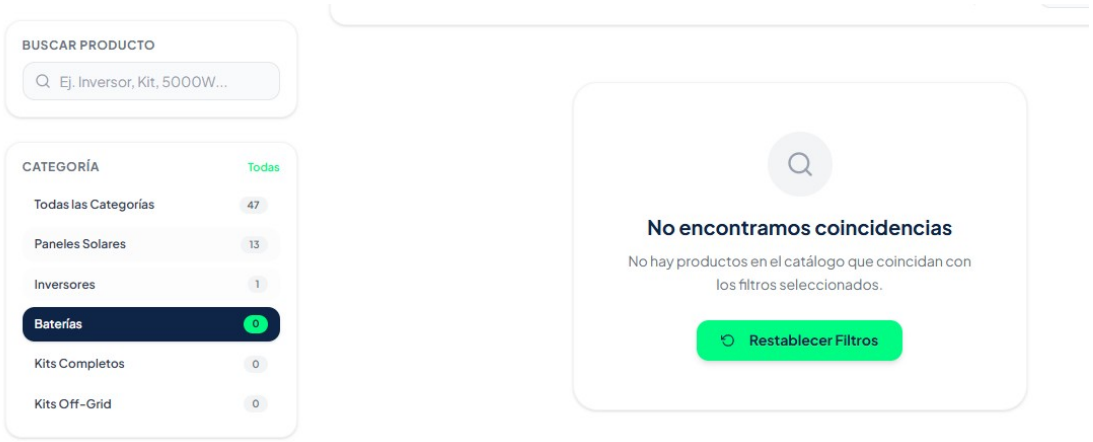
Anexo 14 – FE-02: Filtro “Paneles Solares” (13 productos).



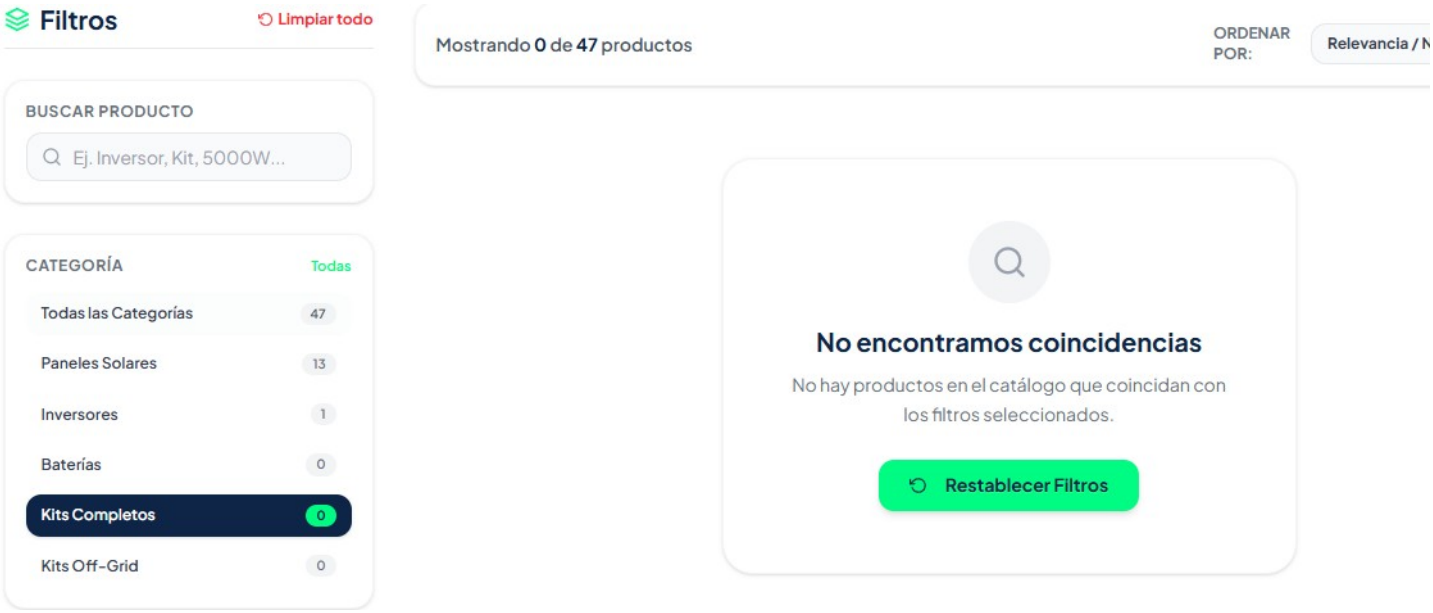
Anexo 15 – FE-02: Filtro “Inversores” (1 producto).



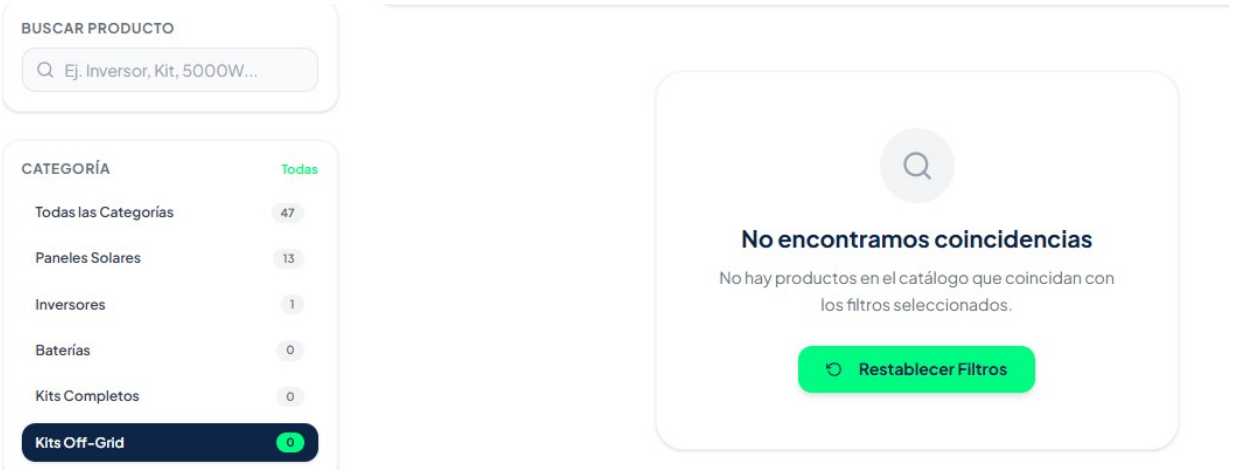
Anexo 16 – FE-02 / DP-11: Filtro “Baterías” sin productos.



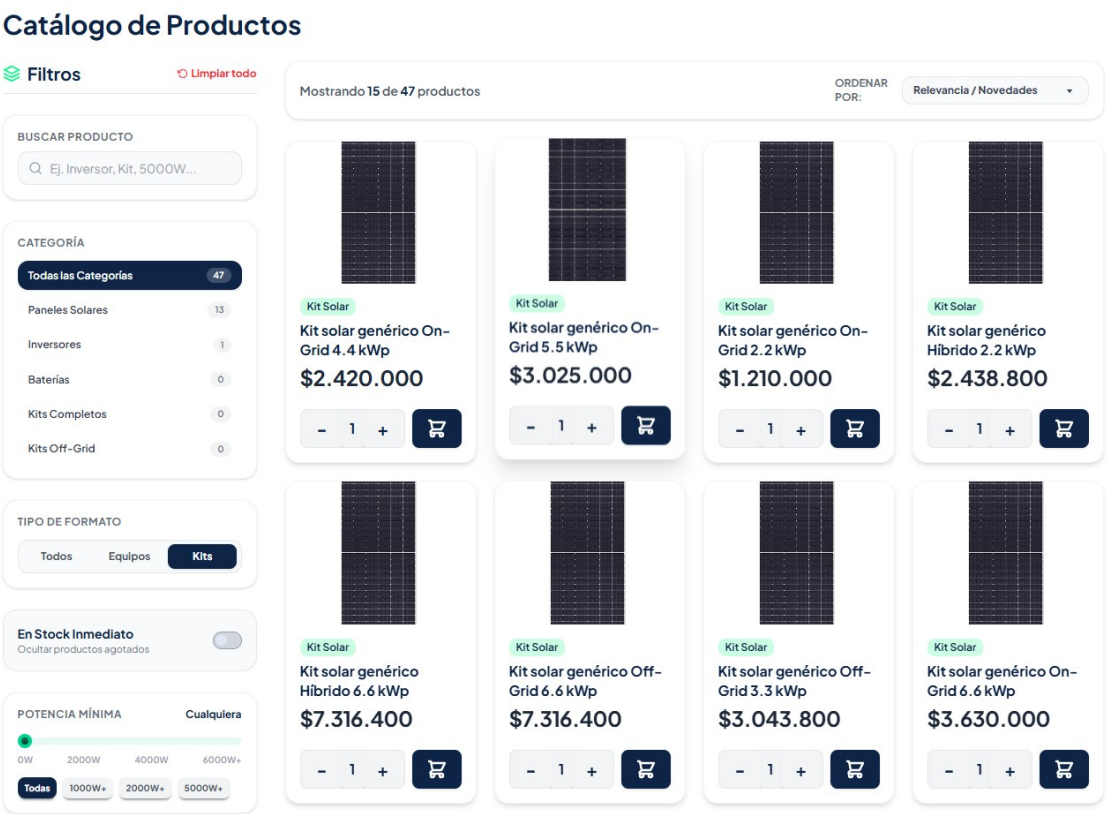
Anexo 17 – FE-02 / DP-11: Filtro “Kits Completos” sin productos.



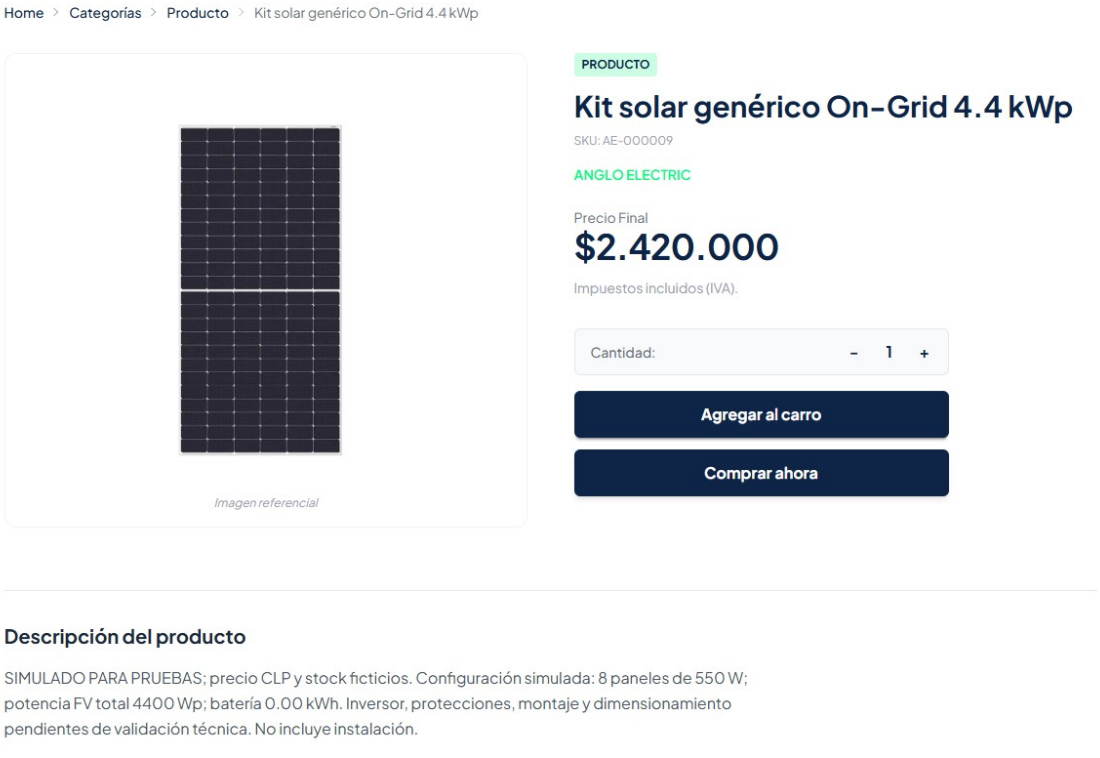
Anexo 18 – FE-02 / DP-11: Filtro “Kits Off-Grid” sin productos.



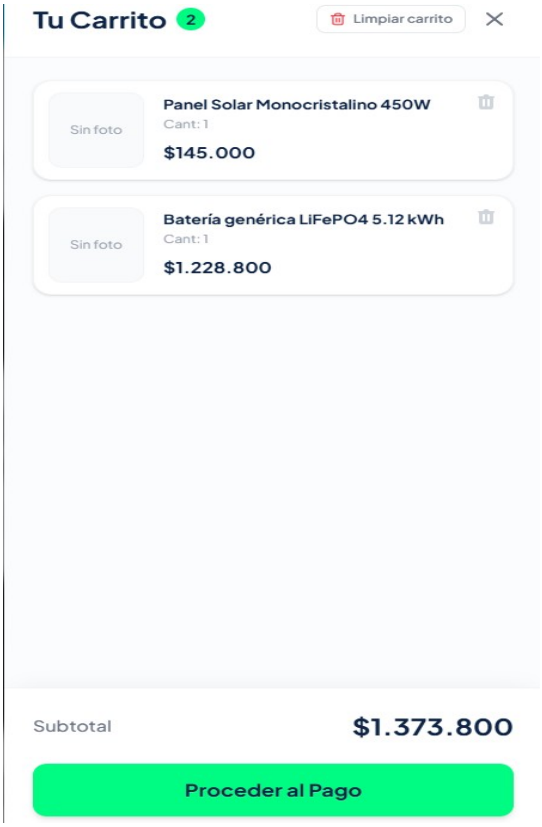
Anexo 19 – FE-02 / DP-11: Filtro “Kits” de Tipo de formato muestra 15 kits sin categoría asignada.



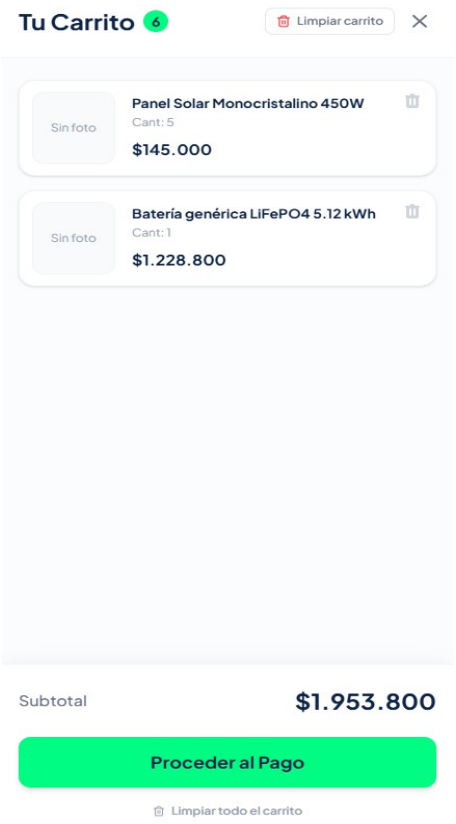
Anexo 20 – FE-03 / DP-14: Página de detalle del producto “Kit solar genérico On-Grid 4.4 kWp”.



Anexo 21 – FE-04: Carrito con 2 productos y subtotal de $1.373.800.



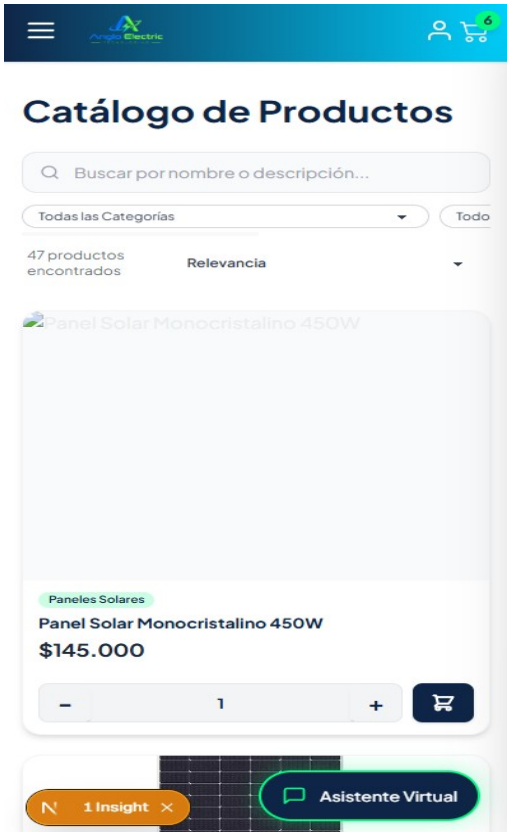
Anexo 22 – FE-04 / DP-15: Carrito recalculado al aumentar la cantidad a 5 unidades (subtotal $1.953.800).



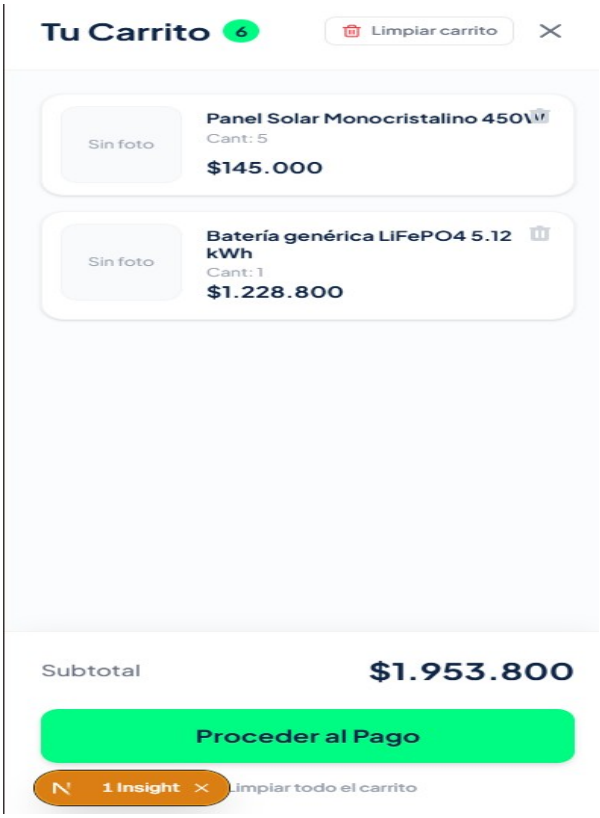
Anexo 23 – FE-05: Validación del formulario de registro: campos obligatorios, bloqueo de números en el nombre y requisitos de contraseña.



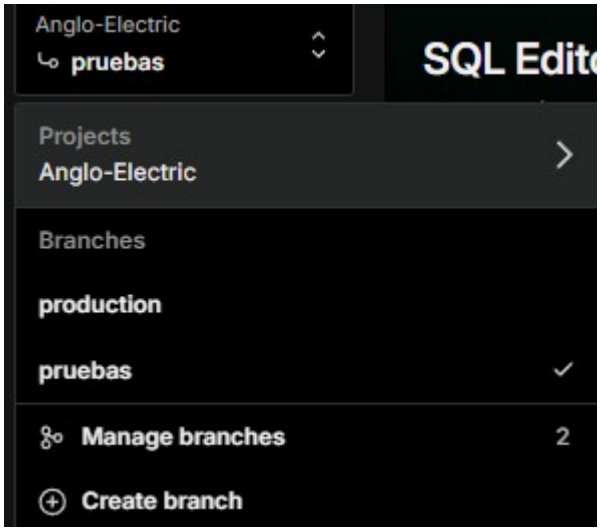
Anexo 24 – FE-06: Catálogo en vista móvil (375 px).



Anexo 25 – FE-06 / DP-18: Carrito en vista móvil con superposición del nombre y el ícono de eliminar.



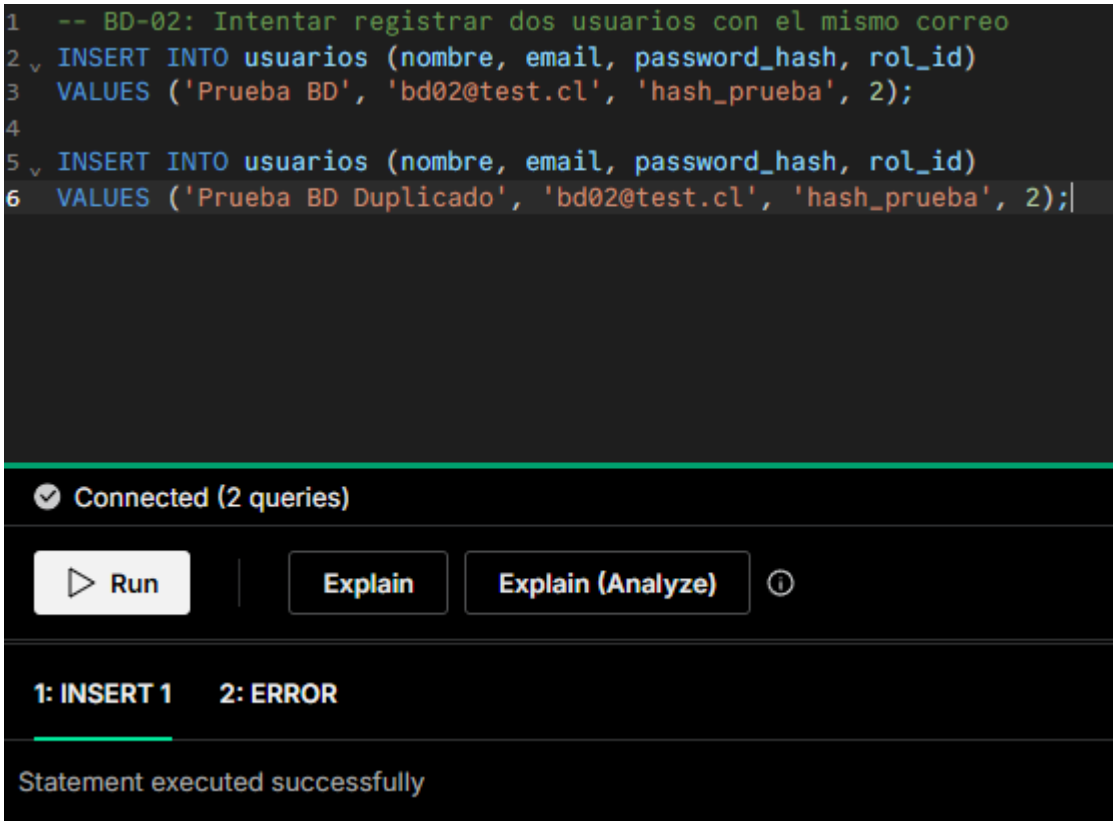
Anexo 26 – Parte C: Rama "pruebas" creada en Neon para ejecutar las pruebas de base de datos.



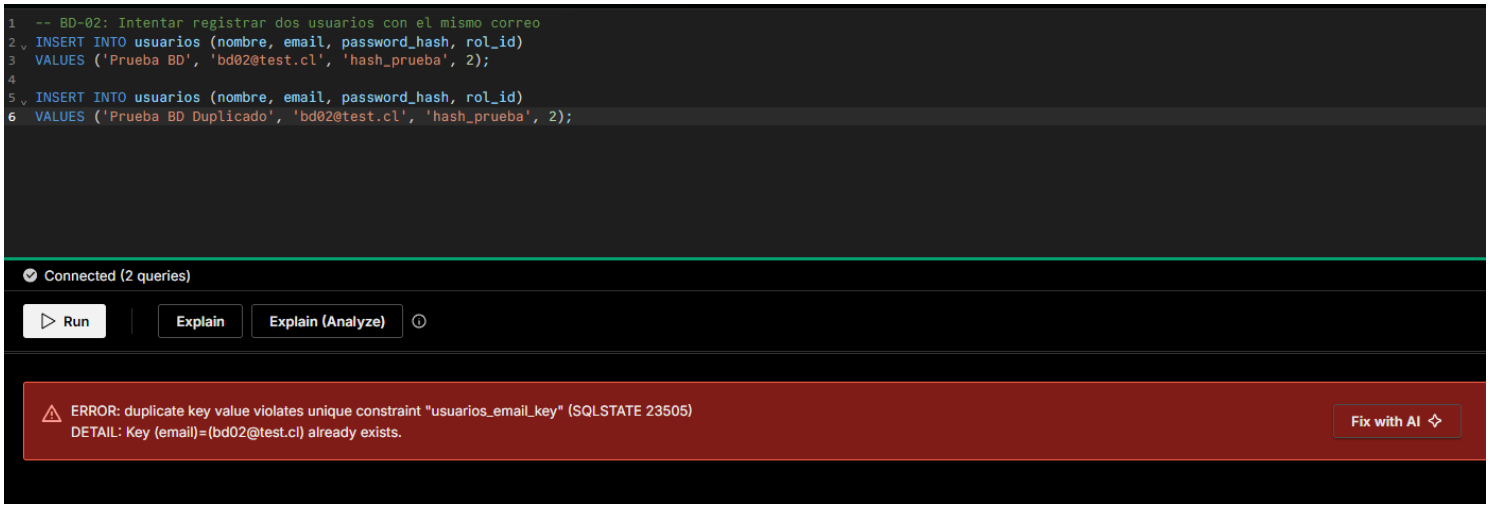
Anexo 27 – BD-01 / DP-20: Resumen de tablas y columnas de la base de datos.

#	tabla	total_columnas	columnas
1	agenda	4	id, pedido_id, fecha_instalacion, estado_agenda
2	categorias	3	id, nombre, descripcion
3	citas_visita	7	id, turno_id, usuario_id, direccion, comuna, telefono, estado
4	costos_productos	2	producto_id, costo_unitario
5	cotizaciones	7	id, usuario_id, consumo_kwh_mensual, comuna, presupuesto_estimado, estado, fecha_solicitud
6	cotizaciones_knn_detalle	7	id, cotizacion_id, clave, huella, entrada, resultado, creado_en
7	detalle_pedidos	5	id, pedido_id, producto_id, cantidad, precio_unitario
8	pedidos	5	id, usuario_id, monto_total, estado, fecha_pedido
9	pedidos_registro	7	pedido_id, clave, huella, cotizacion_id, subtotal, entrega, partidas
10	productos	10	id, nombre, descripcion, precio, stock, potencia_watts, categoria_id, imagen_url, creado_en, es_kit
11	roles	2	id, nombre
12	turnos_visita	2	id, inicio
13	usuarios	7	id, nombre, email, password_hash, telefono, rol_id, creado_en
14	vista_kpis_financieros	9	id_pedido, fecha_venta, producto, cantidad, precio_unitario, ingreso_total, costo_base_unitario, ganancia_neta, margen_porcentaje

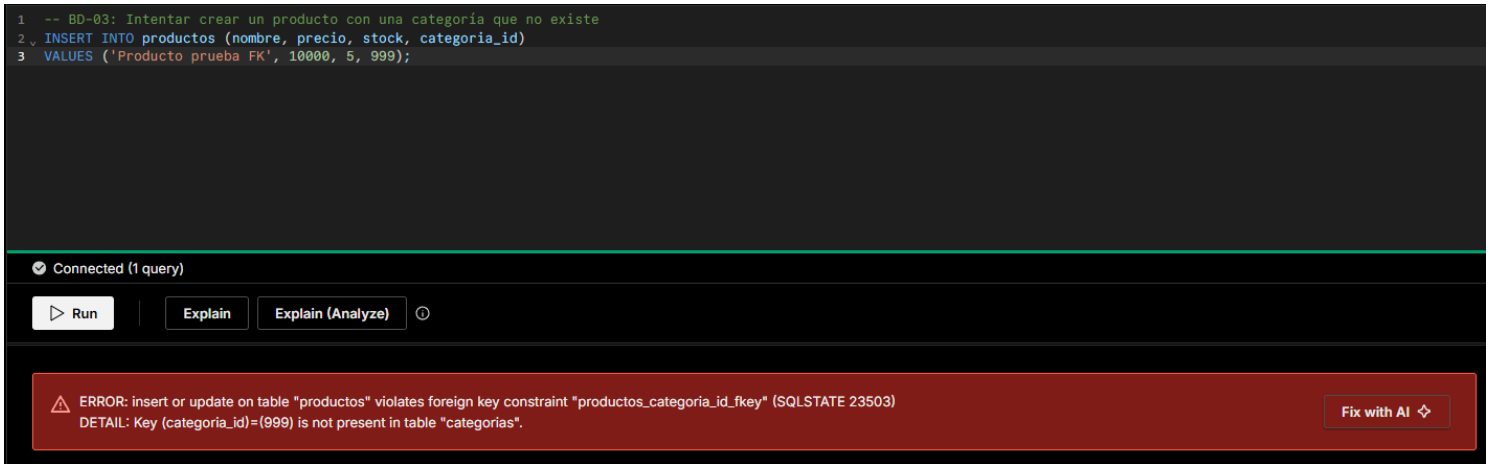
Anexo 28 – BD-02: Primer registro insertado correctamente.



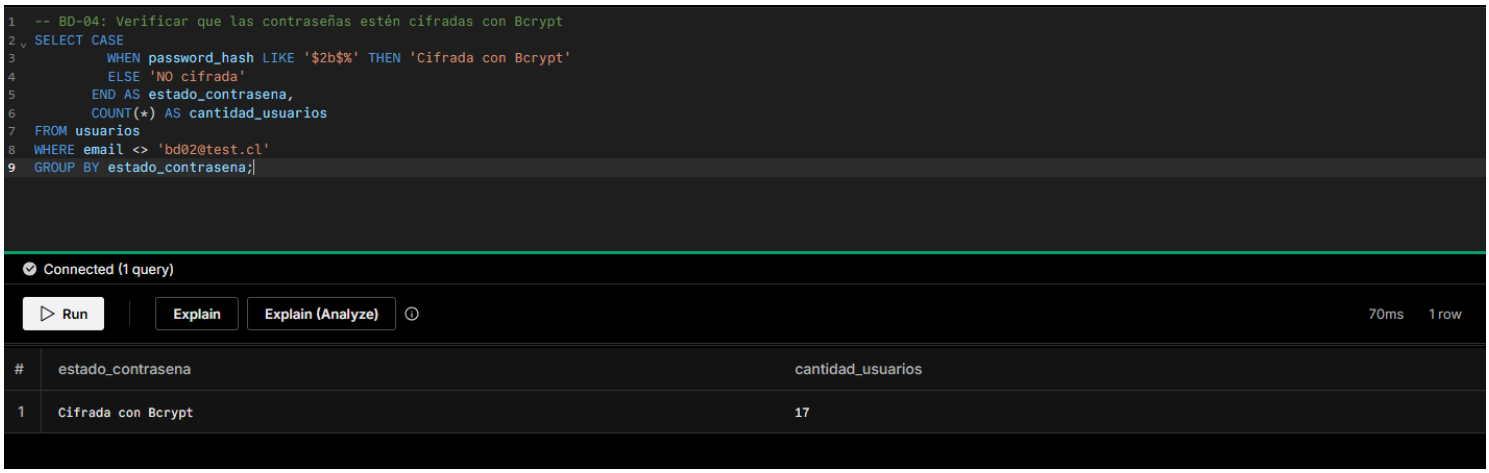
Anexo 29 – BD-02: Segundo registro rechazado por restricción UNIQUE de email.



Anexo 30 – BD-03: Inserción rechazada por clave foránea inexistente (categoria_id = 999).



Anexo 31 – BD-04: 17 usuarios con contraseña cifrada con Bcrypt.



Anexo 32 – BD-05 / DP-21: Revisión de calidad de datos de la tabla productos.

```sql
-- BD-05: Revisar calidad de los datos de productos
SELECT 'Total de productos' AS revision, COUNT(*) AS cantidad FROM productos
UNION ALL
SELECT 'Sin categoría (categoria_id vacío)', COUNT(*) FROM productos WHERE categoria_id IS NULL
UNION ALL
SELECT 'Sin descripción', COUNT(*) FROM productos WHERE descripcion IS NULL OR descripcion = ''
UNION ALL
SELECT 'Sin potencia (potencia_watts vacío)', COUNT(*) FROM productos WHERE potencia_watts IS NULL
UNION ALL
SELECT 'Sin imagen', COUNT(*) FROM productos WHERE imagen_url IS NULL OR imagen_url = ''
UNION ALL
SELECT 'Stock en 0', COUNT(*) FROM productos WHERE stock = 0
UNION ALL
SELECT 'Stock negativo', COUNT(*) FROM productos WHERE stock < 0
UNION ALL
```

Connected (1 query)

Run | Explain | Explain (Analyze) — 71ms — 9 rows

#	revision	cantidad
1	Total de productos	47
2	Sin categoría (categoria_id vacío)	33
3	Sin descripción	6
4	Sin potencia (potencia_watts vacío)	12
5	Sin imagen	46
6	Stock en 0	6
7	Stock negativo	0
8	Precio en 0 o negativo	0
9	Marcados como datos de prueba	40

Anexo 33 – BD-06 / DP-22: Ejecución de docker compose up; solo se levantan PostgreSQL local y pgAdmin.

```
Símbolo del sistema - docker
anglo_postgres  | waiting for server to start....2026-11-13 18:59:01.658 UTC [48] LOG:  starting PostgreSQL 16.16 (Debian 16.16-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64
-bit
anglo_postgres  | 2026-11-13 18:59:01.660 UTC [48] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
anglo_postgres  | 2026-11-13 18:59:01.664 UTC [51] LOG:  database system was shut down at 2026-11-13 18:58:58 UTC
anglo_postgres  | 2026-11-13 18:59:01.668 UTC [48] LOG:  database system is ready to accept connections
anglo_postgres  |  done
anglo_postgres  | server started
anglo_postgres  | CREATE DATABASE
anglo_postgres  |
anglo_postgres  |
anglo_postgres  | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
anglo_postgres  |
anglo_postgres  | 2026-11-13 18:59:01.833 UTC [48] LOG:  received fast shutdown request
anglo_postgres  | waiting for server to shut down....2026-11-13 18:59:01.834 UTC [48] LOG:  aborting any active transactions
anglo_postgres  | 2026-11-13 18:59:01.836 UTC [48] LOG:  background worker "logical replication launcher" (PID 54) exited with exit code 1
anglo_postgres  | 2026-11-13 18:59:01.836 UTC [49] LOG:  shutting down
anglo_postgres  | 2026-11-13 18:59:01.837 UTC [49] LOG:  checkpoint starting: shutdown immediate
anglo_postgres  | 2026-11-13 18:59:01.906 UTC [49] LOG:  checkpoint complete: wrote 935 buffers (5.7%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.024 s, sync=0.042 s, total=0.071 s; sync files=301, l
ongest=0.002 s, average=0.001 s; distance=4346 kB, estimate=4346 kB; lsn=0/1943068, redo lsn=0/1943068
anglo_postgres  | 2026-11-13 18:59:01.913 UTC [48] LOG:  database system is shut down
anglo_postgres  |  done
anglo_postgres  | server stopped
anglo_postgres  |
anglo_postgres  | PostgreSQL init process complete; ready for start up.
anglo_postgres  |
anglo_postgres  | 2026-11-13 18:59:01.958 UTC [1] LOG:  starting PostgreSQL 16.16 (Debian 16.16-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
anglo_postgres  | 2026-11-13 18:59:01.958 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
anglo_postgres  | 2026-11-13 18:59:01.958 UTC [1] LOG:  listening on IPv6 address "::", port 5432
anglo_postgres  | 2026-11-13 18:59:01.961 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
anglo_postgres  | 2026-11-13 18:59:01.965 UTC [64] LOG:  database system was shut down at 2026-11-13 18:59:01 UTC
anglo_postgres  | 2026-11-13 18:59:01.969 UTC [1] LOG:  database system is ready to accept connections
anglo_pgadmin   | /venv/lib/python3.15/site-packages/sshtunnel.py:1044: SyntaxWarning: 'return' in a 'finally' block
anglo_pgadmin   |   return (ssh_host,
anglo_pgadmin   | /venv/lib/python3.15/site-packages/alembic/operations/batch.py:298: UserWarning: Unnamed CHECK constraint on reflected table 'sharedserver' is being omitted from the table recreate; pass the
constraint explicitly using batch_alter_table.table_args.  Naming CHECK constraints avoids this going forward.
anglo_pgadmin   |   util.warn(
anglo_pgadmin   | NOTE: Configuring authentication for SERVER mode.
anglo_pgadmin   |
anglo_pgadmin   | pgAdmin 4 - Application Initialisation
anglo_pgadmin   | ======================================
anglo_pgadmin   |
anglo_pgadmin   | postfix/postlog: starting the Postfix mail system
anglo_pgadmin   | [2026-11-13 18:59:17 +0000] [1] [INFO] Starting gunicorn 23.0.0
anglo_pgadmin   | [2026-11-13 18:59:17 +0000] [1] [INFO] Listening at: http://[::]:80 (1)
anglo_pgadmin   | [2026-11-13 18:59:17 +0000] [1] [INFO] Using worker: gthread
anglo_pgadmin   | [2026-11-13 18:59:17 +0000] [125] [INFO] Booting worker with pid: 125
anglo_pgadmin   | /venv/lib/python3.15/site-packages/sshtunnel.py:1044: SyntaxWarning: 'return' in a 'finally' block
anglo_pgadmin   |   return (ssh_host,

View in Docker Desktop  View Config  Enable Watch  Detach
```

Anexo 34 – BD-06 / DP-22: Docker Desktop muestra solo 2 contenedores en ejecución: anglo_postgres (5432) y anglo_pgadmin (5050).

